SEGUNDO CÉREBRO

Desenvolvimento de Aplicações Web Modernas com Inteligência Artificial

Guia de estudo e referência prática

Objetivo

Construir um segundo cérebro especializado em desenvolvimento web, capaz de apoiar o estudo de programação, arquitetura de aplicações, ferramentas de IA, segurança, bancos de dados, versionamento, deploy e boas práticas — com base em fontes selecionadas e validação humana.

Material de apoio elaborado a partir do tema e das fontes selecionadas para o notebook. Este PDF é um guia organizado de estudo, não uma exportação literal do relatório gerado pelo NotebookLM.

Sumário

- 1. Fundamentos da web: HTML, CSS e JavaScript
- 2. TypeScript e React
- 3. Backend moderno com Supabase
- 4. Git, GitHub e deploy com Vercel
- 5. Segurança web e OWASP Top 10
- 6. IA aplicada à programação: coding agents e Agent Skills
- 7. Validação, testes, acessibilidade e performance
- 8. Fluxo de trabalho recomendado
- 9. Perguntas para revisar o aprendizado
- 10. Fontes selecionadas

Como usar este guia

Estude um capítulo por vez. Consulte a documentação original quando precisar de detalhes, porque ferramentas e versões mudam. Ao usar IA, confirme afirmações importantes nas fontes, teste o código e mantenha registro das decisões.

1. Fundamentos da web: HTML, CSS e JavaScript

HTML define a estrutura e o significado do conteúdo: títulos, parágrafos, botões, imagens, formulários e regiões semânticas. Escolher elementos semânticos adequados ajuda navegadores e tecnologias assistivas a interpretar a página.

CSS controla a apresentação visual, incluindo cores, tipografia, espaçamento e layout. Flexbox costuma ser útil para organizar elementos em uma dimensão; CSS Grid facilita composições em linhas e colunas. Media queries ajudam a adaptar a interface a diferentes tamanhos de tela.

JavaScript adiciona comportamento: responder a cliques, validar formulários, atualizar a interface e comunicar-se com APIs. A interação com o usuário deve ser testada em diferentes cenários, inclusive erros e dados inesperados.

> Exemplo conceitual: HTML organiza o formulário; CSS torna o formulário legível e responsivo; JavaScript trata o envio e apresenta mensagens de sucesso ou erro.

2. TypeScript e React

TypeScript acrescenta um sistema de tipos ao JavaScript. Os tipos ajudam a identificar certas inconsistências durante o desenvolvimento, melhoram a experiência do editor e tornam contratos de dados mais explícitos. Eles não eliminam todos os bugs nem substituem testes em execução.

React permite construir interfaces a partir de componentes reutilizáveis. Componentes recebem dados por props e podem manter estado quando a interface precisa acompanhar mudanças. A composição ajuda a dividir telas grandes em partes menores e mais fáceis de testar.

Boas práticas iniciais:

- Divida a interface em componentes com responsabilidades claras.
- Defina tipos para os dados e propriedades relevantes.
- Trate estados de carregamento, sucesso, vazio e erro.

- Evite duplicar lógica e mantenha componentes compreensíveis.
- Consulte a documentação da versão efetivamente utilizada no projeto.

3. Backend moderno com Supabase

Supabase oferece serviços como banco de dados PostgreSQL, autenticação, APIs e armazenamento de arquivos. Em aplicações reais, é importante compreender onde os dados ficam, quem pode acessá-los e como a interface se comunica com o backend.

> Segurança essencial: nunca coloque chaves secretas de servidor no código entregue ao navegador. Configure permissões e políticas de acesso — incluindo Row Level Security (RLS) quando aplicável — e teste os acessos com diferentes tipos de usuário. Não presuma que esconder um botão na interface protege os dados no servidor.

Antes de publicar, verifique as políticas de leitura, criação, atualização e exclusão, as regras de autenticação, o tratamento de erros e a exposição de arquivos armazenados.

4. Git, GitHub e deploy com Vercel

Git registra o histórico de alterações do código. GitHub hospeda repositórios e facilita colaboração, revisão e automação. Commits pequenos e descritivos ajudam a entender o que mudou e por quê.

Vercel pode automatizar builds e deploys a partir de um repositório conectado, incluindo ambientes de pré-visualização. Antes de considerar um deploy pronto, teste a URL publicada, formulários, rotas, variáveis de ambiente, permissões e comportamento em dispositivos móveis.

Checklist de publicação:

- Confirmar o comando de build e a pasta de saída do projeto.
- Configurar variáveis de ambiente no serviço de hospedagem, não no código público.
- Testar a versão de preview antes de publicar em produção.
- Verificar rotas, links, imagens, console do navegador e acessibilidade básica.
- Manter um procedimento de correção ou rollback quando a plataforma oferecer esse recurso.

5. Segurança web e OWASP Top 10

A segurança precisa fazer parte da arquitetura desde o início. O OWASP Top 10 é um recurso de conscientização sobre riscos importantes de aplicações web; ele não é, sozinho, uma auditoria completa nem garante que um sistema esteja seguro.

Práticas fundamentais:

- Aplicar autorização no servidor e o princípio do menor privilégio.
- Validar dados de entrada e usar consultas parametrizadas quando apropriado.
- Proteger sessões e implementar autenticação de acordo com o risco do sistema.
- Manter dependências atualizadas e investigar alertas de vulnerabilidade.
- Não registrar nem publicar tokens, senhas, chaves de API ou outros segredos.
- Revisar configurações de produção e mensagens de erro que possam revelar informações.

> Consulte a versão oficial do OWASP Top 10 e a documentação de segurança específica das tecnologias do projeto. A lista e a nomenclatura de riscos podem mudar entre edições.

6. IA aplicada à programação: coding agents e Agent Skills

Assistentes de programação podem explicar conceitos, sugerir código, resumir erros e apoiar tarefas repetitivas. Coding agents podem executar sequências de tarefas com ferramentas, arquivos e contexto do projeto, conforme as permissões e capacidades do ambiente.

Agent Skills são instruções e recursos especializados que ajudam um agente a seguir um fluxo de trabalho ou atuar em um domínio específico. A estrutura e o suporte variam entre plataformas; verifique sempre a documentação oficial de Claude Code, Lovable ou Base44 antes de presumir que um formato ou comportamento seja idêntico.

A IA pode produzir código plausível, mas incorreto, usar APIs inexistentes, ignorar requisitos ou introduzir vulnerabilidades. Portanto:

- Dê contexto, objetivo, limites e critérios de aceitação claros.
- Peça mudanças pequenas e revisáveis em vez de alterações enormes sem controle.
- Exija explicação das decisões e dos arquivos modificados.
- Confira a documentação oficial das APIs e bibliotecas sugeridas.
- Execute testes e revise segurança, acessibilidade e tratamento de erros.
- Não conceda acesso desnecessário a dados, segredos ou sistemas.

7. Validação, testes, acessibilidade e performance

Código gerado por IA deve ser tratado como uma proposta que precisa de revisão. Testes unitários verificam partes isoladas; testes de integração verificam a interação entre partes; testes de ponta a ponta simulam fluxos importantes do usuário. A estratégia depende do tamanho e do risco da aplicação.

Acessibilidade inclui HTML semântico, navegação por teclado, rótulos de formulário, textos alternativos adequados e contraste suficiente. Performance pode envolver imagens dimensionadas corretamente, redução de JavaScript desnecessário, carregamento eficiente e medição em condições realistas.

Checklist rápido antes de entregar:

- O fluxo principal funciona do começo ao fim?
- Há estados de carregamento, vazio, sucesso e erro?
- Os dados são validados e as permissões foram testadas?
- A interface funciona em telas pequenas e com teclado?
- Os links, imagens e rotas funcionam no ambiente publicado?
- O código foi revisado e os testes relevantes foram executados?

8. Fluxo de trabalho recomendado

1. Planejar: Defina o problema, os usuários, as funcionalidades necessárias e os critérios de aceitação.

2. Prototipar: Use IA para explorar estrutura e interface, sem tratar a primeira saída como definitiva.

3. Implementar: Construa componentes, tipos e integrações em etapas pequenas.

4. Integrar backend: Modele os dados, configure autenticação e políticas de acesso.

5. Validar: Execute testes, revise acessibilidade, segurança e performance.

6. Versionar: Registre mudanças com commits claros e revise diferenças.

7. Publicar: Faça deploy, teste a aplicação publicada e monitore problemas.

9. Perguntas para revisar o aprendizado

1. Qual é a diferença entre a função de HTML, CSS e JavaScript?

2. Que tipo de problema o sistema de tipos do TypeScript ajuda a detectar? O que ele não garante?

3. Como componentes, props e estado se relacionam no React?

4. Por que as permissões do backend precisam ser verificadas no servidor?

5. Qual é a diferença entre Git e GitHub?

6. Quais verificações devem ocorrer antes de publicar uma aplicação?

7. Por que uma resposta de IA precisa ser revisada e testada?

8. Como instruções especializadas ou Agent Skills podem ajudar um agente — e quais riscos devem ser considerados?

10. Fontes selecionadas

MDN Web Docs — fundamentos de HTML, CSS e JavaScript
https://developer.mozilla.org/pt-BR/

React — documentação para aprender React
https://react.dev/learn

TypeScript — Handbook
https://www.typescriptlang.org/docs/handbook/intro.html

GitHub Docs — documentação da plataforma
https://docs.github.com/pt

Supabase Docs — banco de dados, autenticação e APIs
https://supabase.com/docs

Vercel Docs — deploy e infraestrutura
https://vercel.com/docs

OWASP Top 10 — riscos de segurança de aplicações web
https://owasp.org/www-project-top-ten/

GitHub Copilot Docs — desenvolvimento assistido por IA
https://docs.github.com/en/copilot

Anthropic — Agent Skills
https://www.anthropic.com/research/skills

Anthropic Engineering — Agent Skills
https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

Base44 — site oficial; confirmar a página específica de Skills no momento da consulta
https://base44.com/

Nota de transparência: este guia é uma síntese de estudo produzida para organizar os temas do projeto. Não reproduz palavra por palavra o relatório do NotebookLM nem substitui a consulta às fontes originais. Confirme versões, recursos e recomendações nas documentações oficiais, especialmente em temas de segurança e ferramentas de IA.